NanoGPT Foundation Model Research Report

June 2026

1 Task 1.3 – Data Cleaning

The dataset was cleaned before training to improve consistency and reduce noisy samples. The preprocessing steps included:

- Deduplication
- Lowercasing and normalization
- Removing excessive punctuation
- Formatting cleanup
- Token normalization

Example transformation:
Dirty Text:

```
YO!!!! $$$ moneyyyy raininggggg!!!!
rapperrrrr in da clubbbbb!!!!
```

Cleaned Text:

```
yo moneyyyy raininggggg
rapperrrrr in da clubbbbb
```

The dirty dataset reached a very low loss quickly because the repetitive noisy patterns were easy for the model to memorize:

- Dirty Dataset Final Validation Loss: 0.1216
- Clean Dataset Final Validation Loss: 0.2218

Even though the dirty dataset achieved a lower loss, the generated outputs were repetitive and overfit to the training data. The clean dataset produced outputs that were more structured and generalized better.

Reflection

This experiment showed that data quality has a major impact on model behavior. A stronger architecture alone cannot fully compensate for poor or inconsistent data. Cleaner data improved stability, coherence, and overall generation quality.

2 Task 2.1 – Character Tokenizer

A character-level tokenizer was implemented using a simple character-to-index mapping.

Results:

- Vocabulary Size: 1747
- Model Parameters: 3.67M
- Final Validation Loss: 2.4330

The model learned spelling patterns and local structures quite well, but long-range semantic understanding was still limited. Character-level generation handled unseen words better, although the outputs were generally slower and less coherent.

3 Task 2.2 – BPE Tokenizer

A GPT-2 style Byte Pair Encoding tokenizer was used for comparison.

Dataset statistics:

- GPT-2 Vocabulary Size: 50257
- Poems Unique Tokens: 39577
- Rap Dataset Unique Tokens: 28093

Tokenizer Comparison

Aspect	Character Tokenizer	BPE Tokenizer
Learning Speed	Slower	Faster
Vocabulary Granularity	Character-level	Subword-level
Generalization	Better for unseen words	Better semantic compression
Generation Quality	Less coherent	More fluent and coherent
Context Efficiency	Poor	More efficient

Analysis Questions

1. Which learns faster?
The BPE tokenizer learned faster because subword tokens capture more semantic information in fewer steps.

2. Which generalizes better?
Character tokenization generalized better for rare spellings and unseen words.

3. Which produces more coherent generations?
The BPE tokenizer generated more coherent and readable outputs.

4. What changed mechanistically?
Instead of learning individual character transitions, the model started learning semantic relationships between larger token units and phrases.

4 Task 3.1 – Token Embeddings

The embedding layer was initialized randomly and visualized before and after training using PCA.

Observations:

- Before training, embeddings looked randomly scattered.
- After training, semantically related tokens moved closer together.
- Frequently used stylistic patterns formed tighter clusters.

Question: What changed in representation space?

Training organized the embedding space into meaningful regions. Tokens appearing in similar contexts developed similar vector representations.

5 Task 3.2 – Positional Encoding

Positional embeddings were implemented and later removed for an ablation study.

Results without positional encoding:

- Validation Loss after 299 steps: 6.0288

Without positional information, the generated text lost proper ordering and became repetitive.

Reflection

Transformers process all tokens simultaneously, so positional embeddings are necessary to preserve sequence order and context flow.

6 Task 3.3 – Self-Attention

Query, Key, and Value projections were implemented along with causal masking.

Experiments included:

- Attention visualization
- Mask removal ablation

After removing masking, the generated text became unstable and repetitive:

> "The stars in the sky The moonlight of the sun with the wind over morning and stable moonlight..."

Question: What information is attention routing?

Attention routed contextual information such as syntax, rhythm, semantic relationships, and long-range dependencies between tokens.

7 Task 3.4 – Multi-Head Attention

A comparison was performed between single-head and multi-head attention models.

Results:

- Single-head Validation Loss: 6.0758
- Multi-head attention generated more stable and diverse outputs.

Reflection

Different attention heads appeared to focus on different relationships such as sentence structure, rhyme patterns, and contextual dependencies.

8 Task 3.5 – Feed Forward Networks

The Feed Forward Network size was reduced aggressively to study its impact.

Results:

- Reduced FFN Validation Loss: 7.2285

Reducing FFN capacity noticeably weakened semantic consistency and output quality.

Question: Why is attention alone insufficient?

Attention mainly routes information between tokens, while FFNs perform nonlinear transformations and increase representational capacity. Without strong FFNs, the model loses expressive power.

9 Task 3.6 – Residual Connections and LayerNorm

Residual connections and LayerNorm were removed separately to analyze training failures.

Results:

- No Residuals Validation Loss: 7.2432
- No LayerNorm Validation Loss: 7.1333

Removing residuals made optimization unstable and slowed convergence. Removing LayerNorm increased variance during training and reduced stability.

Failure Analysis

Ablation	Observed Failure
No Residuals	Gradient instability and weak convergence
No LayerNorm	Unstable optimization and higher variance
Reduced FFN	Weak semantic representations

No Positional Encoding	Loss of sequence structure
No Masking	Information leakage and repetition

10 Task 3.7 – Full Transformer Block

The final transformer block combined:

- Multi-head self-attention
- Feed Forward Network
- Residual connections
- Layer normalization
- Positional embeddings

The complete architecture trained stably and generated more coherent outputs compared to the ablated versions.

11 Task 4.1 – First Training Run

The baseline NanoGPT model was trained on both poem and rap datasets.

Poem dataset results:

- Final Train Loss: 4.9184
- Final Validation Loss: 5.2068
- GPU Usage: $\sim$ 1115 MB

Rap dataset results:

- Final Train Loss: 4.5426
- Final Validation Loss: 4.9488

The rap dataset trained slightly better because of its repetitive stylistic structure and stronger local patterns.

12 Task 4.2 – Hyperparameter Sensitivity

Learning rate experiments:

Learning Rate	Final Validation Loss
0.001	6.2682
0.0003	6.8661
0.00005	Very slow convergence

Reflection Questions

1. Which parameter mattered most?
Learning rate had the biggest effect on convergence speed and training stability.

2. Which hurt training most?
Very small learning rates slowed training significantly.

3. Which improved quality most?
A balanced learning rate combined with larger model capacity improved generation quality the most.

13 Task 4.3 – Scaling Experiments

Three model sizes were tested.

Configuration	Parameters	Validation Loss
Tiny (2 Layers, 128 Dim)	6.86M	6.4393
Medium (4 Layers, 256 Dim)	16.09M	6.5451
Large (6 Layers, 384 Dim)	30.04M	Improved fluency observed

Reflection

Larger models produced more fluent and contextually consistent text. Scaling improved representation learning and long-range dependency modeling, although it also increased memorization and computational cost.

14 Task 5.1 – Decoding Experiments

Different decoding methods were tested:

- Temperature Sampling
- Top-k Sampling
- Top-p Sampling
- Loop Penalization

Observations:

- Higher temperature increased diversity but reduced coherence.
- Top-k sampling produced more stable outputs.
- Top-p sampling balanced diversity and readability well.
- Greedy decoding often resulted in repetitive loops.

Reflection

The same model behaves differently because decoding strategies change how probabilities are sampled during generation.

15 Task 6.1 – Failure Mode Analysis

The model was stress tested for:

- Hallucinations
- Repetition loops
- Context forgetting
- Memorization
- Contradictions

Example repetitive failure:

"I get money to to to to..."

Failure Taxonomy

Failure Type	Cause
Hallucinations	Weak semantic grounding
Repetition Loops	Decoding instability during inference
Context Forgetting	Limited context window
Memorization	Repetitive training data
Contradictions	Weak long-range reasoning

16 Part 8 – Final Reflection

1. What is a Foundation Model?

A Foundation Model is a large pretrained neural network trained on broad datasets that can adapt to many downstream tasks through prompting or fine-tuning.

2. Why do transformers scale?

Transformers scale efficiently because attention mechanisms allow parallel computation and capture long-range relationships effectively.

3. Why do models hallucinate?

Models hallucinate because they are optimized for next-token prediction rather than factual accuracy.

4. Why does data matter more than expected?

Data quality strongly influences representation learning, memorization behavior, and generalization capability.

5. Why does prompting work?

Prompts guide the probability distribution of the model and activate different learned contextual patterns.

6. Why does the same model behave differently across runs?

Randomness in sampling and decoding strategies changes token selection paths during generation.

7. What are the structural limits of decoder-only transformers?

Decoder-only transformers struggle with long-term reasoning, factual consistency, and memory efficiency.

8. Which failure mode surprised you most?

Repetition loops were the most surprising because they appeared even when the model loss seemed reasonable.

9. What would you improve next?

Future improvements could include larger datasets, longer training, stronger regularization, better decoding strategies, and retrieval-based augmentation.